Bulle d'R. (Re)Découvrir Quarto sous R

Sandrine LYSER

15 octobre 2026

Table des matières

Jeu de données utilisé dans cette session . . . 2

1 Introduction générale **2**
1.1 Le problème de départ… . . . 2
1.2 Les questions associées… . . . 2
1.3 La programmation lettrée . . . 3
1.4 Programmation classique *vs* programmation lettrée . . . 3

2 Présentation de Quarto **4**
2.1 Quarto en une idée . . . 4
2.2 Utiliser Quarto dans RStudio . . . 4
2.2.1 1. Installer les outils . . . 4
2.2.2 2. Créer un document . . . 5
2.2.3 3. Compiler (rendre) un document . . . 5
2.3 Le fichier `.qmd` . . . 6
2.3.1 Exemple . . . 6

3 Utiliser Quarto **7**
3.1 Écrire simplement avec Markdown . . . 7
3.1.1 Syntaxe **Markdown** pour la mise en forme du texte . . . 8
3.2 Le YAML . . . 8
3.2.1 Définir les paramètres du document et du rendu . . . 8
3.2.2 Écrire le YAML . . . 9
3.2.3 Personnaliser les rendus . . . 9
3.2.4 Où placer les réglages YAML ? . . . 9
3.3 Produire un rapport : Word ou pdf . . . 10
3.4 Créer une diapositive . . . 11
3.5 Ajouter un bloc de code R . . . 11
3.5.1 Paramétrer les blocs de code . . . 11
3.5.2 Nommer les blocs de code . . . 12
3.5.3 Cas particulier des blocs de codes des tableaux et figures . . . 12
3.6 Ajouter des tableaux . . . 13
3.6.1 Produire un tableau manuellement . . . 13
3.6.2 Produire un tableau avec R . . . 14
3.7 Ajouter une image . . . 14
3.8 Ajouter une équation . . . 15
3.8.1 Équation au sein d'un texte . . . 15
3.8.2 Équation en bloc . . . 15

4 À retenir **15**
4.1 En résumé . . . 15

5 Ressources **16**
5.1 Pour aller plus loin . . . 16

Diffusé sous licence CC BY-NC-SA 4.0

Jeu de données utilisé dans cette session

`hdv2003`

- disponible dans le package `{questionr}`
- échantillon tiré de l'enquête Histoire de vie, réalisée en 2003 en France, par l'Insee, auprès de la population âgée de 18 ans et plus
- 2000 lignes et 20 variables

Dictionnaire des variables

Variable	Description	Type
id	Identifiant unique de l'individu	entier
age	Âge de l'individu	entier
sexe	Sexe de l'individu	facteur (Femme, Homme)
nivetud	Niveau d'études	facteur (8 modalités)
poids	Poids de l'individu dans l'échantillon	numérique
occup	Occupation principale	facteur (7 modalités)
qualif	Qualification professionnelle	facteur (7 modalités)
freres.soeurs	Nombre de frères et soeurs	entier
clso	Classe sociale	facteur (3 modalités)
relig	Religion	facteur (6 modalités)
trav.imp	Importance accordée au travail	facteur (4 modalités)
trav.satisf	Satisfaction au travail	facteur (3 modalités)
hard.rock	Écoute du hard rock	facteur (Oui, Non)
lecture.bd	Lecture de bandes dessinées	facteur (Oui, Non)
peche.chasse	Pratique de la pêche ou de la chasse	facteur (Oui, Non)
cuisine	Pratique de la cuisine	facteur (Oui, Non)
bricol	Pratique du bricolage	facteur (Oui, Non)
cinema	Fréquentation du cinéma	facteur (Oui, Non)
sport	Pratique d'un sport	facteur (Oui, Non)
heures.tv	Nombre d'heures passées devant la télévision	numérique

1 Introduction générale

1.1 Le problème de départ…

Une étude contient

- le code qui transforme les données
- les tableaux et graphiques produits
- le texte qui explique les résultats
- le rapport ou la présentation à partager

Quand ces éléments sont séparés, il faut les mettre à jour manuellement et vérifier qu'ils racontent toujours la même histoire

1.2 Les questions associées…

- Comment éviter les copier-coller des résultats pour que le document Word corresponde bien aux résultats ?
- Comment rendre son travail intelligible pour les collègues (et soi-même) ?

— Comment assurer une certaine reproductibilité du travail ?

— Comment mettre à disposition du code ?

» **Il existe des outils pour résoudre ces problèmes liés au partage et la reproductibilité du travail**

1.3 La programmation lettrée

Idée centrale : associer dans un même document du code, des explications et des résultats

````
Ce script est dédié aux premières analyses des données hdv2003 du package "questionr"

#  Étape 1. Chargement des packages
Dans un premier temps on charge les packages R
  `{r}
  # Packages
  library(tidyverse)  # v2.0.0
  library(questionr)  # v0.8.2
  `

#  Étape 2. Chargement des données
Dans un second temps on charge les données
  `{r}
  # Chargement des données depuis le package "questionr"
  data(hdv2003)
  `

#  Étape 3. Analyses
## Tableau
Puis on crée un premier tableau qui montre la distribution de la variable sexe
  `{r}
  hdv2003 |>
    select(sexe) |>
    questionr::freq(total = TRUE)
  `

## Graphique
Enfin, on crée un graphique, qui illustre la différence du nombre d'heures passées devant la
↪  télévision selon le sexe
  `{r}
  hdv2003 |>
  ggplot() +
    aes(heures.tv) +
    geom_boxplot() +
    facet_wrap(vars(sexe))
  `
````

— Le texte explique **pourquoi** et **comment** l'analyse est réalisée

— Le code est exécuté lors du rendu (`Render`)

— Les sorties, tableaux et graphiques sont insérés automatiquement dans le document final

— Le document reste lisible sans être un simple script commenté

— Modifier les données ou le code peut mettre à jour tout le document

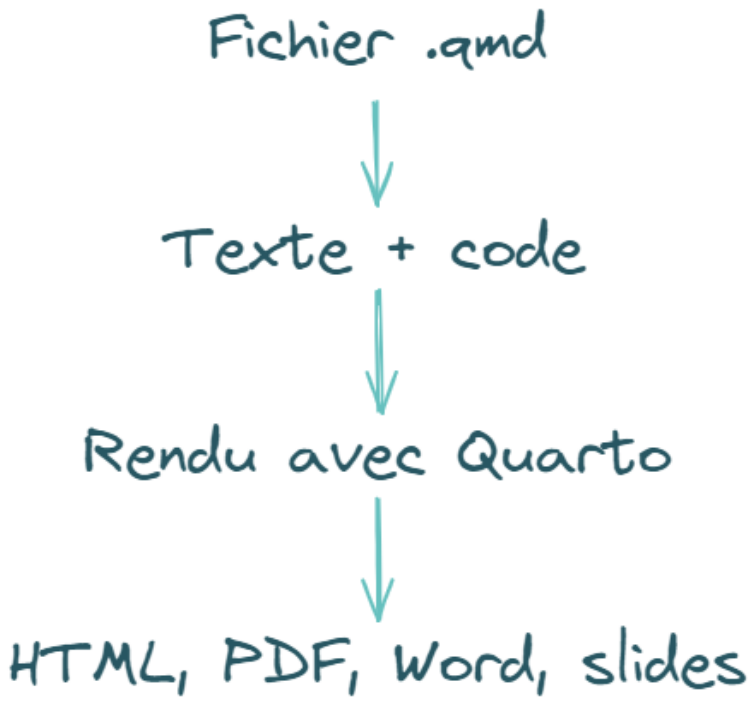
1.4 Programmation classique *vs* programmation lettrée

	Programmation classique	Programmation lettrée
Type de fichier	.R	.rmd, .qmd
Organisation des fichiers	Script et rapport séparés	Code et analyse regroupés
Contenu du fichier	Code prédominant	Texte prédominant
Documentation	Brefs commentaires	Texte explicatif détaillé
Lecture	Orientée code	Orientée documentation
Intégration des résultats	Par copier-coller	Grâce à la compilation
Mise à jour des tables/plots	Manuelle	Lors de la compilation
Reproductibilité	Limitée	Facilitée

» **Quarto applique ce principe de programmation lettrée**

2 Présentation de Quarto

2.1 Quarto en une idée



Un seul fichier qui rassemble

- du texte rédigé en **Markdown**
- du code **R** et/ou dans d'autres langages
- les résultats produits par ce code

et qui est compilé pour générer un document, un site ou une présentation

» **Le fichier source au format `.qmd` est modifiable**
» **Le rendu, qui peut prendre plusieurs formats de sortie, est le résultat prêt à être diffusé et partagé**

2.2 Utiliser Quarto dans RStudio

2.2.1 1. Installer les outils

- Installer **R**
- Installer **RStudio**

— Installer l'extension **Quarto** dans **RStudio** si nécessaire

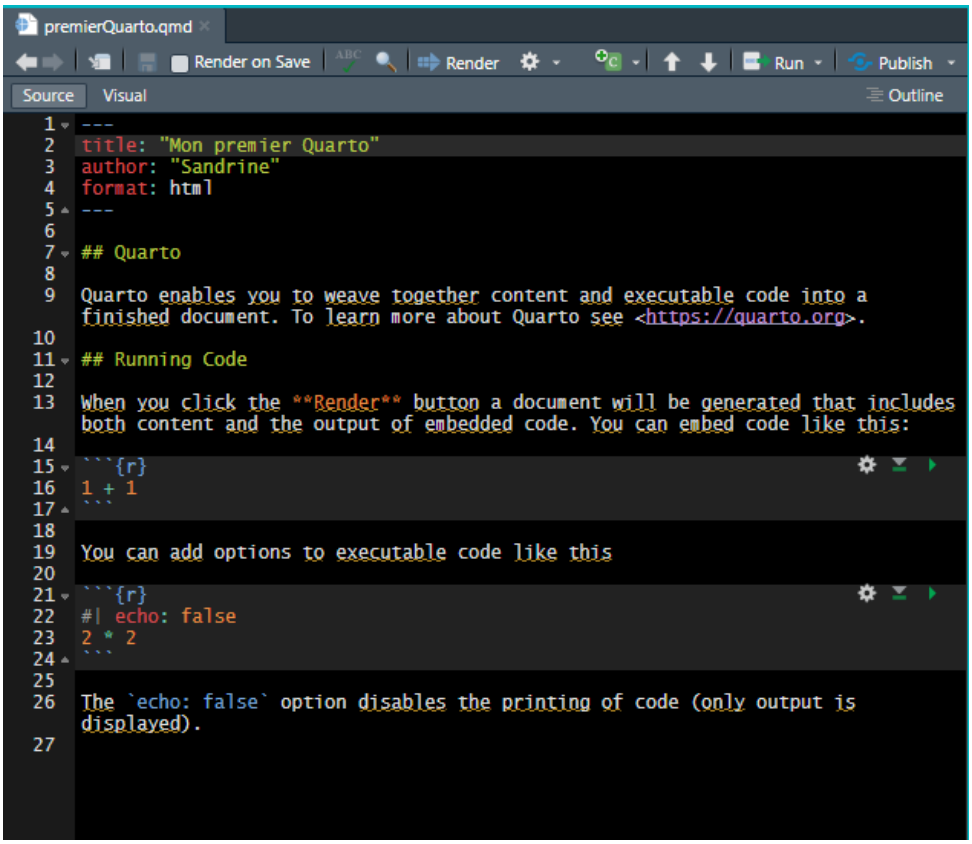
RStudio propose une interface intégrée pour créer, modifier et compiler des documents **Quarto**

2.2.2 2. Créer un document

Dans **RStudio**

1. **File New File Quarto Document…**
2. Choisir un titre, un auteur ou une autrice, et éventuellement le format de sortie
3. Choisir **html**, **pdf**, **Word** ou une présentation
4. Cliquer sur **Create**

» **RStudio** crée un fichier `.qmd` contenant déjà un YAML, du texte et un exemple de bloc de code



2.2.3 3. Compiler (rendre) un document

1. Ouvrir le fichier `.qmd` créé dans **RStudio**
2. Modifier le titre dans le **YAML**
3. Cliquer sur **Render**
4. Consulter le résultat dans l'onglet **Viewer** ou dans le navigateur

Le bouton **Render** exécute le document

— il lance le code **R**

— il insère les résultats

— et il produit le fichier de sortie choisi

2.3 Le fichier `.qmd`

Un document **Quarto** contient généralement trois types de contenu

— **Des métadonnées** : titre, format, options

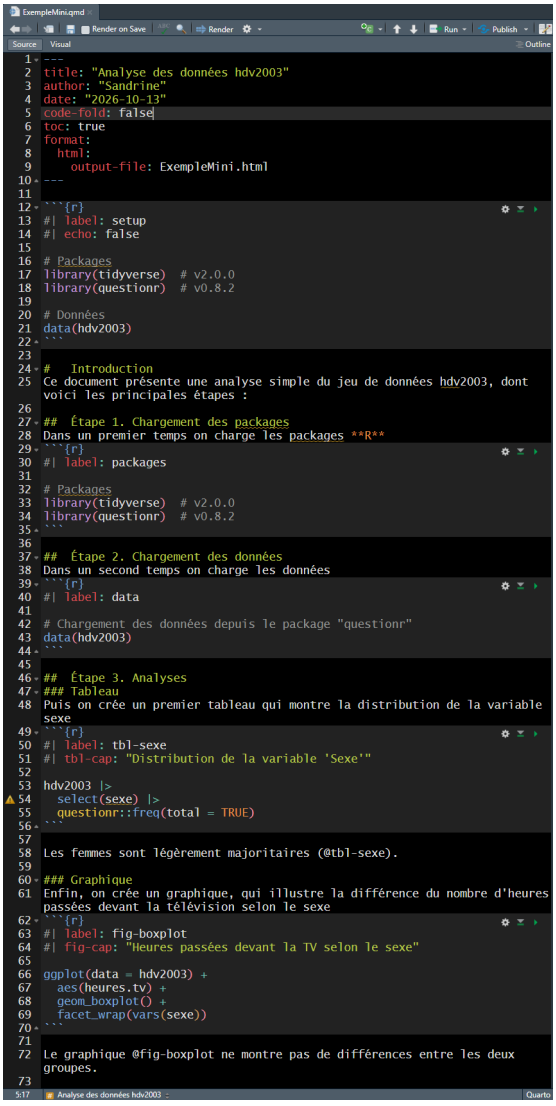
— **Du texte** : titres, paragraphes, listes, liens

— **Du code** : blocs **R** exécutés à la compilation (au rendu)

L'extension `.qmd` signifie "Quarto Markdown"

2.3.1 Exemple

2.3.1.1 Script Quarto (.qmd)



➡

2.3.1.2 Rendu en html



Analyse des données hdv2003

AUTEUR·RICE
Sandrine

DATE DE PUBLICATION
2026-10-13

Introduction

Ce document présente une analyse simple du jeu de données hdv2003, dont voici les principales étapes :

Étape 1. Chargement des packages

Dans un premier temps on charge les packages **R**

```
# Packages
library(tidyverse)  # v2.0.0
library(questionr)  # v0.8.2
```

Étape 2. Chargement des données

Dans un second temps on charge les données

```
# Chargement des données depuis le package "questionr"
data(hdv2003)
```

Étape 3. Analyses

Tableau

Puis on crée un premier tableau qui montre la distribution de la variable sexe

Tableau 1. Distribution de la variable 'Sexe'

```
hdv2003 |>
  select(sexe) |>
  questionr::freq(total = TRUE)
```

```
         n   % val%
Homme  899  45   45
Femme 1101  55   55
Total 2000 100  100
```

Les femmes sont légèrement majoritaires (tableau 1).

Graphique

Enfin, on crée un graphique, qui illustre la différence du nombre d'heures passées devant la télévision selon le sexe

```
ggplot(data = hdv2003) +
  aes(heures.tv) +
  geom_boxplot() +
  facet_wrap(vars(sexe))
```


Figure 1. Heures passées devant la TV selon le sexe

Le graphique figure 1 ne montre pas de différences entre les deux groupes.

Réutilisation

CC BY-NC-SA 4.0

Table des matières

- Introduction
  - Étape 1. Chargement des packages
  - Étape 2. Chargement des données
  - Étape 3. Analyses
    - Tableau
    - Graphique

Autres formats

- PDF
- RevealJS (inrae)

3 Utiliser Quarto

3.1 Écrire simplement avec Markdown

— L'écriture sous **Quarto** s'appuie sur la syntaxe **Markdown**
» Les marqueurs **Markdown** sont des conventions simples que **Quarto** transforme à l'affichage
Les bases en **Markdown**

— On peut écrire du code, des listes, des tableaux, des liens vers des pages web, des images, etc.

— Il est également possible de gérer les citations et les références croisées

Exemple

```
## Une diapositive

Un paragraphe explicatif
```

```
- Premier point
- Deuxième point

**Texte en gras** et *texte en italique*
```

Quarto reprend les principes de **RMarkdown** mais constitue un système plus général, compatible avec plusieurs langages de programmation (**R**, **Python**, **Julia**, **JavaScript**)

3.1.1 Syntaxe Markdown pour la mise en forme du texte

Caractéristique	Syntaxe Markdown	Résultat
italic	`texte en *italique*`	texte en *italique*
bold	`texte en **gras**`	texte en **gras**
superscript	`superficie de 200 m^2^`	superficie de 200 m^2
subscript	`Les individus notés n~i~`	Les individus notés n_i
headings	`# Titre 1`	# Titre 1
	`## Titre 2`	## Titre 2
	`### Titre 3`	### Titre 3
	`#### Titre 4`	#### Titre 4
	`##### Titre 5`	##### Titre 5
	`###### Titre 6`	###### Titre 6
links	Site `[Quarto](https://quarto.org/)`	Site **Quarto**
visible links	le lien est `<https://quarto.org/>`	le lien est https://quarto.org
quote	`> ceci est important à retenir`	> ceci est important à retenir

Il n'est pas possible de souligner du texte en Markdown, le soulignement étant réservé aux liens hypertextes

3.2 Le YAML

3.2.1 Définir les paramètres du document et du rendu

— Dans un fichier `.qmd`, le bloc YAML se trouve généralement au début du document, entre deux lignes de trois tirets `---`

— Il donne à **Quarto** des informations sur le document et indique le format de sortie
  — les métadonnées : titre, auteur, date, langue, etc.

  — les options sur les formats de sortie : `html`, `pdf`, `word`, `revealjs`, etc.
  — les options de mise en page : table des matières, apparence, etc.

```yaml
---
title: "Découvrir Quarto sous R"
author: "Nom et prénom"
date: "2026-10-13"
lang: fr
format:
  revealjs:
    theme: simple
    slide-number: true
---
```

Le YAML est le premier élément évalué lors de la compilation
» En cas d'erreur, la compilation s'arrête quasi instantanément

---

3.2.2 Écrire le YAML

- `title` : le titre du document
- `author` : le nom de l'auteur ou de l'autrice
- `date` : la date affichée
- `lang` : la langue du document, ici le français
- `format` : le type de sortie demandé
- `revealjs` : des diapositives `html` avec `revealjs`
- `theme` et `slide-number` : des options de présentation

La syntaxe est très sensible à l'indentation
» Ajouter 2 espaces pour chaque niveau d'indentation

Dans l'exemple précédent, les options placées sous `revealjs` appartiennent à ce format

> **Voir la documentation Quarto pour plus d'informations sur les nombreuses options disponibles**
>
> - section Front matter
> - pour le format HTML
> - pour le format PDF
> - pour le format Word
> - pour le format Revealjs
> - pour le format Powerpoint

3.2.3 Personnaliser les rendus

Le YAML permet de régler le thème et le numéro des diapositives, ou encore la création d'une table des matières ou l'utilisation d'une base bibliographique (au format `.bib`)

```yaml
format:
  revealjs:
    theme: simple
    slide-number: true
  pdf:
    toc: true
bibliography: mon_fichier.bib
```

Les thèmes prédéfinis incluent notamment `simple`, `dark`, `solarized` et `beige`. Il est possible de créer son propre thème en le personnalisant avec le `css`

Mieux vaut commencer par quelques options utiles et ajouter le reste selon les besoins

---

3.2.4 Où placer les réglages YAML ?

Les réglages **YAML** peuvent être spécifiés dans un fichier séparé pour éviter

- de répéter dans chaque `.qmd` les options communes au projet
- un long bloc **YAML** en début de script

Règle pratique

- les options communes dans `_quarto.yml`

— les spécificités dans le `.qmd`

Niveau	Fichier	Portée
Document	YAML en tête du `.qmd`	Ce document seulement
Dossier	`_metadata.yml`	Documents du dossier
Projet	`_quarto.yml`	Tous les documents du projet

Exemple

```yaml
# _quarto.yml
project:
  type: default

format:
  revealjs:
    theme: simple
    slide-number: true

execute:
  echo: true
```

3.3 Produire un rapport : Word ou pdf

Le même fichier `.qmd` peut produire une page html, une présentation, un document Word ou un pdf

» **Bonne pratique :** rédiger et maintenir le `.qmd`, puis générer le format selon les besoins

Exemple

```yaml
---
title: "Rapport d'analyse"
author: "Nom et prénom"
date: today
lang: fr
format:
  docx:
    toc: true
  pdf:
    toc: true
    number-sections: true
---
```

Comment lire ce YAML ?

— `docx` produit un fichier Word modifiable

— `pdf` produit un document pdf non modifiable
— `toc: true` ajoute une table des matières au document

— `number-sections: true` numérote les sections dans le pdf

</> Commandes à exécuter dans un terminal, pour obtenir le rendu dans un format précis

```
quarto render rapport.qmd --to docx
quarto render rapport.qmd --to pdf
```

Le rendu PDF nécessite généralement une distribution **LaTeX**, par exemple **TinyTeX**, pensez à vérifier que c'est installé sur votre machine

3.4 Créer une diapositive

Avec `revealjs`, 2 balises # successives (##) créent les diapositives

```
## Première diapositive

Contenu de la première diapositive

## Deuxième diapositive

Contenu de la deuxième diapositive
```

Une seule balise # sert à créer des sections avec une diapositive de titre

3.5 Ajouter un bloc de code R

- Un bloc **R** exécutable commence par ```` ```{r} ```` et se termine par trois accents graves (*back ticks*)
- Dans le rendu, **Quarto** peut inclure le code, le résultat, ou les deux selon les options choisies
- Les options commencent par le *hashpipe* `#|` et s'écrivent en utilisant la syntaxe : `option: value`

````
```{r}
# Distribution de la variable "age"
hdv2003 |>
  select(age) |>
  summary()
```
````

```
      age
 Min.   :18.00
 1st Qu.:35.00
 Median :48.00
 Mean   :48.16
 3rd Qu.:60.00
 Max.   :97.00
```

Dans cet exemple, le code et le résultat sont affichés car le bloc de code est écrit avec les options

- `#| echo: true` pour afficher le code
- `#| eval: true` pour afficher le résultat

- **R n'est pas le seul langage possible dans les blocs de code**
  » On spécifie le langage souhaité dans les accolades {} sur la première ligne du bloc de code, à la suite des "'
- **Pour afficher directement le résultat d'un calcul dans une phrase, on insère directement le code dans la phrase, en utilisant un seul *back tick* ' pour encadrer le code**
  - **Syntaxe** : L'âge moyen des '`{r} hdv2003 |> nrow()`' observations de hdv2003 est égal à '`{r} hdv2003 |> pull(age) |> mean(na.rm = TRUE)`'
  - **Résultat** : L'âge moyen des 2000 observations de hdv2003 est égal à 48.157

3.5.1 Paramétrer les blocs de code

Les options des blocs de code s'écrivent avec le préfixe `#|` et permettent de contrôler l'exécution, l'affichage du code et la mise en forme des sorties

TABLEAU 2. Principales options des blocs de code

Option	Description
`label`	Donner un nom au bloc (utile pour le référencer et y faire appel dans le texte)
`eval: true/false`	Exécuter le code du bloc
`echo: true/false`	Afficher le code dans le document final
`include: true/false`	Contrôler l'inclusion du code et/ou des résultats dans le document final (`false` : supprime tout)
`warning: true/false`	Inclure les avertissements générés dans le document final
`message: true/false`	Afficher dans le document final, les messages générés par le code

---

3.5.2 Nommer les blocs de code

Chaque bloc de code peut recevoir un **label**

— qui doit être un nom composé de chiffres et de lettres, court, explicite et **unique** (les tirets - et les *underscores* _ sont autorisés, en revanche les espaces et caractères spéciaux sont déconseillés)

— doit être placé de préférence sur la première ligne du bloc de code, avant les autres options

````
```{r}
#| label: datahdv
#| eval: true
data(hdv2003)
```
````

Il permet d'identifier le bloc dans le document et dans les messages d'erreur et de faciliter la lecture et la maintenance d'un document long

---

3.5.3 Cas particulier des blocs de codes des tableaux et figures

3.5.3.1 Référencer tableaux et figures dans le texte

— On donne des noms de labels au bloc de code avec un préfixe spécifique : `tbl-` pour les tableaux ; `fig-` pour les figures (images, graphiques)
— On peut donner une légende avec les options `#| tbl-cap:` pour les tableaux ; `#| fig-cap:` pour les figures
— On fait ensuite référence au tableau ou à la figure avec `@prefixe-nom` dans le texte

3.5.3.1.1 Bloc de code

````
```{r}
#| label: fig-sexeTV-demo
#| fig-height: 2
#| fig-width: 3
#| fig-cap: "Relation entre heures passées devant la TV et sexe des individus."

hdv2003 |>
  ggplot() +
  aes(x = sexe, y = heures.tv) +
  geom_boxplot()
```
````

3.5.3.1.2 Résultat du bloc de code



FIGURE 1. Relation entre heures passées devant la TV et sexe des individus.

3.5.3.1.3 Référence dans le texte

```
Comme illustré par la @fig-sexeTV, il y a peu de différences entre les groupes
```

3.5.3.1.4 Rendu

Comme illustré par la figure 1, il y a peu de différences entre les groupes

---

3.5.3.2 Paramétrer le rendu des blocs de code des graphiques R

Plusieurs options permettent d'améliorer le rendu des graphiques **R**

Option	Rôle	Exemple
`label`	Identifie le bloc et permet une référence croisée	`#| label: fig-age`
`fig-cap`	Ajoute une légende à la figure	`#| fig-cap: "Distribution de la variable 'age'"`
`fig-width`	Définit la largeur, en pouces, de la figure	`#| fig-width: 7`
`fig-height`	Définit la hauteur, en pouces, de la figure	`#| fig-height: 4.5`
`fig-align`	Définit l'alignement horizontal (`left`, `center` ou `right`)	`#| fig-align: center`
`fig-alt`	Ajoute un texte alternatif pour les personnes qui ne peuvent pas voir la figure	`#| fig-alt: "Diagramme en boîte des heures quotidiennes passées devant la télévision selon le sexe"`
`dpi`	Définit la résolution de l'image produite	`#| dpi: 300`

3.6 Ajouter des tableaux

Un tableau peut être écrit directement en **Markdown** ou produit par **R**

3.6.1 Produire un tableau manuellement

- Les tableaux directement écrits en **Markdown** sont
  - adaptés aux petits tableaux fixes
  - simples à rédiger et à modifier
- L'alignement dans les cellules est contrôlé avec les `:`
  Voir la page d'aide Quarto dédiée pour les options disponibles

⚠ Les valeurs ne sont pas recalculées lors du rendu

Syntaxe

```
| Colonne 1    | Colonne 2    | Colonne 3    | Colonne 4    |
|:-------------|:------------:|-------------:| ------------ |
| Alignement   | Alignement   | Alignement   | Alignement   |
| à gauche     | centré       | à droite     | par défaut   |
: Titre du tableau.
```

Résultat

TABLEAU 3. Titre du tableau.

Colonne 1	Colonne 2	Colonne 3	Colonne 4
Alignement à gauche	Alignement centré	Alignement à droite	Alignement par défaut

Vous ne savez pas comment écrire un tableau en Markdown ?
» Utilisez l'outil en ligne

3.6.2 Produire un tableau avec R

- Les tableaux produits par un bloc de code **R**
  - sont recalculés à chaque rendu
  - sont cohérents avec les données et le code
  - peuvent être référencés dans le texte grâce au label `tbl-`
  - peuvent être décrits par une légende avec l'option `#| tbl-cap:`
- Les tableaux sont rendus propres et lisibles grâce à la fonction `kable()` du package {`knitr`}

```
```{r}
#| label: tbl-hdv
#| tbl-cap: "Extrait des données hdv2003."
hdv2003 |>
  select(id:qualif) |>
  slice(1:2) |>
  knitr::kable()
```
```

TABLEAU 4. Extrait des données hdv2003.

id	age	sexe	nivetud	poids	occup	qualif
1	28	Femme	Enseignement superieur y compris technique superieur	2634.398	Exerce une profession	Em-ploye
2	23	Femme	NA	9738.396	Etudiant, eleve	NA

3.7 Ajouter une image

Une image non générée directement dans un bloc de code **R** peut être insérée directement en **Markdown** avec la syntaxe suivante
`![Texte](chemin/image.extension)`

- le texte écrit dans les crochets est la légende de l'image
- le chemin indiqué dans les parenthèses est relatif au fichier `.qmd`
  Prévoir dans le projet un dossier qui stocke les images, pour faciliter l'organisation (par exemple, `img/` comme ici pour cette présentation)

Syntaxe Markdown
`![Logo Quarto](img/quarto.png)`

Résultat



FIGURE 2. Logo Quarto

3.8 Ajouter une équation

L'écriture d'équations mathématiques est possible avec la syntaxe **LaTeX**

- en ligne en encadrant l'équation par `$`
- en "bloc", centré au milieu de la page, en encadrant l'équation par `$$`

3.8.1 Équation au sein d'un texte

Syntaxe

```
La célèbre équation d'Einstein est $E = mc^2$.
```

Résultat
La célèbre équation d'Einstein est $E = mc^2$.

3.8.2 Équation en bloc

Syntaxe

```
La célèbre équation d'Einstein est

$$
E = mc^2
$$
```

Résultat
La célèbre équation d'Einstein est

$$E = mc^2$$

4 À retenir

4.1 En résumé

Les différents éléments pour utiliser **Quarto** sous **R**

- `.qmd` : le fichier source **Quarto**
- **YAML** : les métadonnées et réglages du document
- **Markdown** : la structure et le texte
- Blocs **R** : le code exécuté et ses résultats
- `render` : la commande qui produit la présentation

5 Ressources

5.1 Pour aller plus loin

De nombreuses ressources en ligne

- Tutoriel **Markdown**
- Tutoriel **Quarto**
- Documentation Quarto : présentations
- Guide Quarto pour Reveal.js
- Documentation Quarto : options d'exécution
- Documentation Quarto : projets et `_quarto.yml`

et plus encore dans la documentation officielle **Quarto**